UrbanFlow: Urban Traffic Graph Optimization Engine

Final Review

October 15, 2026

Done By: Group 9
AMAN ANTONY (MUT24CA013)
JENNIFER JOFFY (MUT24CA042)
JOHN VARGHESE (MUT24CA044)
SANGEETHA SHALOM SAJI (MUT24CA065)

Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Existing System
5. Proposed System
6. Software and Hardware Requirements
7. System Architecture
8. Modules
9. Core Algorithms
10. Techniques
11. Work Progress
12. Implementation and Results
13. SDG Addressed
14. Conclusion
15. Future Scope
16. References

Introduction

- **Urban traffic congestion:** Increasing vehicle numbers and limited road capacity lead to longer travel times, fuel consumption, and pollution in cities.
- **Graph-based modelling:** UrbanFlow represents road junctions as vertices and road segments as edges, forming a weighted graph of the transportation network.
- **Traffic simulation:** The system estimates how vehicles distribute themselves across available routes based on travel time and road congestion.
- **Network-level analysis:** A change in one road can affect traffic on several connected roads, so the entire network must be considered.
- **Infrastructure evaluation:** The system allows road widening, road conversion scenarios to be tested before real-world implementation.
- **Real-world application:** The project applies its graph-based simulation pipeline to road-network data from Kochi obtained through OpenStreetMap.

Problem Statement

To develop a graph-based urban traffic simulation and optimization system that models road networks using vertices and edges, utilizes real-world road data from OpenStreetMap, and estimates travel times using the Bureau of Public Roads (BPR) congestion function. The system aims to simulate traffic assignment, analyse congestion, investigate the Braess Paradox, and evaluate road network modifications such as converting roads to one-ways or widening roads and changing road capacities. By applying suitable graph algorithms and comparing metrics such as average travel time, congestion levels, and traffic throughput before and after interventions, UrbanFlow seeks to identify effective infrastructure changes that improve overall traffic flow and network efficiency.

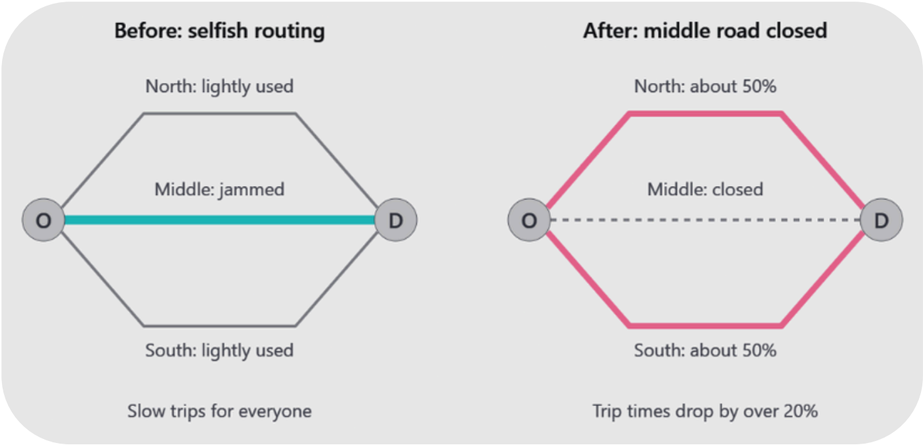
Objectives

- **Develop a weighted graph model:** Represent road junctions as vertices and road segments as edges, storing attributes such as free-flow travel time, road capacity, and traffic volume.
- **Simulate equilibrium traffic flow:** Use the BPR congestion function and the Method of Successive Averages (MSA) to estimate how traffic distributes itself when drivers choose their preferred routes.
- **Identify congestion bottlenecks:** Analyse traffic volume relative to road capacity to identify road segments that are heavily congested or operating beyond capacity.
- **Evaluate road-network interventions:** Simulate road widening, conversion, then compare the modified network with its original baseline.
- **Validate the Braess Paradox:** Reproduce a controlled scenario in which closing a strategically placed road improves overall travel time, demonstrating the importance of network-level analysis.
- **Apply the model to an area (Kochi):** Obtain real road-network data from OpenStreetMap and apply the same graph-building and simulation workflow to a real urban environment.

Braess Paradox

The Braess Paradox is a known result in transportation theory: adding or keeping open a road can raise total travel time. It happens because each driver picks the route that looks fastest to them, which is called User Equilibrium. In UrbanFlow's scenario, three roads connect an origin to a destination, and closing the quick middle road makes the whole network faster.

- User Equilibrium: nobody can improve their own trip by switching routes.
- Why it appears: a quick road attracts too many drivers and becomes the bottleneck, while the steady roads sit underused.
- The result: closing the middle road splits traffic about 50/50 between north and south, and trip times fall by over 20%.
- Takeaway: more capacity does not guarantee better flow, so every change must be tested.



Existing System

1. **Traditional Traffic Planning**
   - Uses traffic surveys, manual observations, historical traffic data, and engineering assessments.
   - Often requires considerable effort to compare several possible infrastructure changes.
   - May not fully capture the indirect effects of a road modification across the entire network.

2. **Navigation and Route Guidance Applications**
   - Recommend routes using estimated travel times and available traffic information.
   - Help individual drivers select routes to reach their destinations efficiently.
   - Are primarily designed for navigation rather than controlled experiments involving road construction or closure.

3. **Conventional Traffic Simulation Tools**
   - Can model traffic flow and evaluate transportation scenarios.
   - May require specialised configuration, detailed datasets, or additional technical resources.
   - Their usefulness depends on the available features, model assumptions, and accessibility.

Existing System (continued)

Limitations of Existing Approaches:

- Infrastructure alternatives can be **expensive** and **time-consuming** to evaluate.
- Improving an individual road **does not necessarily improve** the entire network.
- It can be **difficult to identify counter-productive roads** before making a physical change.
- An accessible environment for graph-based, before-and-after traffic experiments is **useful.**

Proposed System

- **Graph-based road representation:** Convert a road network into a graph containing junctions, road segments, and attributes needed for traffic analysis.
- **Congestion-dependent travel-time calculation:** Apply the BPR function to estimate how road travel times increase as traffic volume approaches or exceeds capacity.
- **Shortest-path route calculation:** Use Dijkstra's algorithm to identify the currently shortest-time routes between origins and destinations.
- **Equilibrium traffic assignment:** Apply MSA to iteratively update traffic distribution until it approaches a stable user-equilibrium condition.
- **Bottleneck identification:** Examine road utilisation and congestion levels to identify segments that may require further investigation.
- **Intervention simulation:** Apply road widening, closure, or addition, rerun the simulation, and compare the results with baseline conditions.
- **Real-world data integration:** Use OpenStreetMap and OSMnx to obtain a real road network, including the Kochi application described in the project.

Software and Hardware Requirements

Software Requirements

- **React, Vite, and TypeScript:** Used to develop the interactive frontend, manage the user interface, and support a structured development workflow.
- **Leaflet / MapLibre:** Map technologies used for displaying road networks and visualising traffic conditions.
- **Python:** Implements the backend logic, traffic models, and graph algorithms.
- **FastAPI**: Provides the API layer through which the frontend communicates with the simulation backend.
- **NetworkX**: Supports graph construction, graph operations, and shortest-path calculations.
- **OpenStreetMap and OSMnx:** Supply and process real-world road-network data.
- **Git:** Supports source-code management and collaboration.

Software and Hardware Requirements (Continued)

Hardware Requirements

- **Computer or laptop:** Required to run the frontend, backend, and simulation.
- **Processor:** A modern processor capable of executing Python and graph-processing operations.
- **RAM:** Sufficient memory to load road-network graphs and perform traffic simulations.
- **Storage:** Required for source code, the Python environment, dependencies, and cached road-network data.
- **Internet connection:** Required when downloading road-network data or installing dependencies; previously cached data can reduce repeated downloads.

System Architechure

1. Frontend Layer
   - Developed using React, Vite, and TypeScript.
   - Provides the interface for selecting road networks and interacting with maps.
   - Displays traffic conditions and the results of scenario comparisons.
2. API Communication Layer
   - Uses FastAPI to connect the frontend with the backend.
   - Receives requests for road-network data and simulation operations.
   - Returns processed information for visualisation.
3. Graph Processing Layer
   - Represents junctions as vertices and roads as edges.
   - Stores road attributes such as free-flow time, capacity, and traffic volume.
   - Uses NetworkX and graph algorithms to analyse routes and network structure.

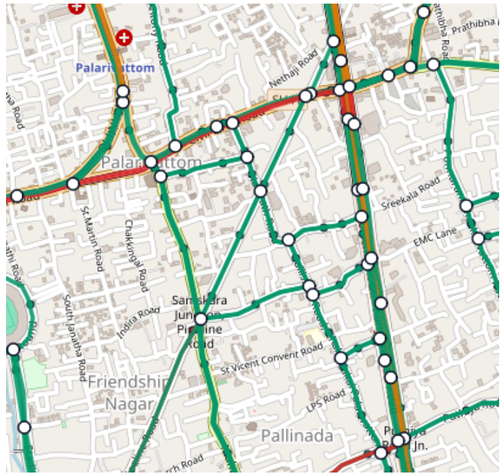
System Architechure (Continued)

4. Traffic Simulation Engine
   - Calculates congestion-dependent travel times using the BPR function.
   - Assigns traffic using shortest-path routing and MSA.
   - Supports bottleneck detection and the evaluation of road modifications.
5. Data Layer
   - Uses OpenStreetMap as a source of real road-network information.
   - Uses OSMnx to retrieve and process road networks.
   - Stores cached data locally to reduce repeated retrieval time.
   - 6. Results and Visualisation Layer
   - Returns simulation metrics to the frontend.
   - Supports comparisons between baseline and modified scenarios.
   - Helps users interpret changes in travel time, congestion, throughput, and connectivity.

Modules

Module 1: Road Network Construction

- Converts road-network information into a graph structure suitable for algorithmic processing.
- Represents each junction as a vertex and each road segment as an edge.
- Associates roads with attributes such as free-flow travel time, capacity, and current traffic volume.
- Provides the graph used by the traffic simulation and route-finding algorithms.



Module 2: Traffic Simulation

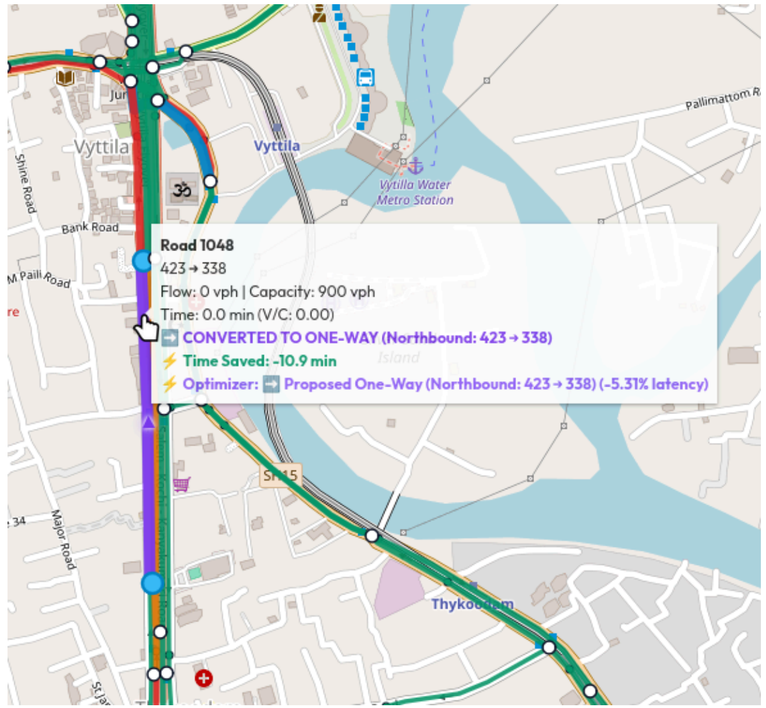
- Calculates road travel times based on traffic volume and available capacity.
- Uses the BPR congestion function to model the effect of increasing traffic.
- Applies Dijkstra's algorithm to identify shortest-time routes.
- Supplies updated travel-time estimates for subsequent traffic assignment iterations.



Modules (Continued)

Module 3: Traffic Equilibrium and Bottleneck Detection

- Uses the Method of Successive Averages to update traffic assignments over repeated iterations.
- Approximates user equilibrium, where drivers cannot improve their travel times by individually switching routes.
- Compares traffic volume with road capacity to identify potentially congested road segments.
- Provides information that can help prioritise road-network investigation.



Module 4: Intervention Analysis

- Allows road-network modifications such as widening, closing, or adding road segments.
- Runs the simulation again after each modification.
- Compares modified-network metrics with the original baseline.
- Helps identify interventions that improve or worsen overall network performance.

Modules (Continued)

Module 5: Real-World Network Integration

- Obtains road-network information from OpenStreetMap.
- Uses OSMnx to process the road data into a graph representation.
- Applies the same simulation workflow to the Kochi road network.
- Uses local caching to reduce the time needed to reload previously retrieved network data.

Core Algorithms

1. Graph Representation
   - The road network is represented using a weighted graph G=(V,E).
   - V represents the set of road junctions, while E represents road segments.
   - Each edge stores relevant attributes, including free-flow travel time, capacity, and traffic volume.
   - This representation enables the use of graph algorithms for routing and network analysis.
2. Dijkstra's Shortest-Path Algorithm
   - Finds the shortest path from a source vertex to reachable vertices in a graph with non-negative edge weights.
   - In UrbanFlow, the weights represent estimated travel times rather than distance alone.
   - The edge costs can change as traffic congestion changes.
   - The algorithm is used to calculate routes based on the current estimated travel times.
   - Basic procedure:
   - Assign a distance of zero to the source and infinity to other vertices.
   - Select the unvisited vertex with the smallest tentative distance.
   - Update the distances of its neighbours when a shorter path is found.
   - Repeat until the required destinations are processed.

Core Algorithms (Continued)

3. BPR Congestion Function
   - UrbanFlow uses the Bureau of Public Roads (BPR) congestion model to estimate travel time under different traffic conditions.
   - Formula: $t = t_o \times [1 + \alpha \times (v/c)^{\beta}]$
   - Where:
   - t = Estimated travel time under current traffic conditions.
   - t_o = Free-flow travel time, when traffic congestion is minimal.
   - v = Traffic volume on the road.
   - c = Road capacity, representing the maximum traffic volume the road can handle.
   - α and β = Model parameters that determine how rapidly travel time increases as traffic volume approaches or exceeds road capacity.
   - The commonly used illustrative values are $\alpha = 0.15$ and $\beta = 4$.
4. Bottleneck Detection
   - Calculates the utilisation of road capacity by comparing traffic volume with available capacity.
   - Roads with high volume-to-capacity ratios are identified for further analysis.
   - The results help locate areas where traffic demand may exceed the available road capacity.

Techniques

1. Graph-Based Network Modelling
   - Represents the urban road system as vertices and edges.
   - Stores the properties required for routing and congestion analysis.
   - Provides a structured model for analysing the relationships between connected roads.
2. Dynamic Edge Weight Calculation
   - Updates estimated road travel times according to traffic conditions.
   - Uses the BPR function to represent increasing delay as road utilisation rises.
   - Allows route choices to respond to changing congestion levels.
3. Shortest-Path Routing
   - Uses Dijkstra's algorithm to find routes with the lowest estimated travel-time cost.
   - Supports route recalculation when edge weights change.
   - Helps simulate drivers selecting routes based on the current network conditions.

Techniques

4. Iterative Traffic Assignment
   - Uses MSA to update traffic distribution over repeated iterations.
   - Incorporates changing road travel times into the assignment process.
   - Approximates user-equilibrium conditions.
5. Scenario-Based Simulation
   - Creates a baseline representing the original road network.
   - Tests modifications such as road closure, widening, or addition.
   - Measures the effects of each change using consistent performance indicators.
6. Real-World Data Processing
   - Uses OpenStreetMap as a source of geographic road information.
   - Uses OSMnx to retrieve and process the road network.
   - Converts the data into a format suitable for graph algorithms and simulation.
7. Caching and Parallel Processing
   - Stores retrieved road-network data to reduce repeated data-loading operations.
   - Uses parallel processing for supported computations across multiple origins.
   - Aims to reduce processing overhead when handling larger networks.

Work Progress

Phase 1: Research and Planning

- Studied urban traffic congestion and the limitations of analysing roads individually.
- Examined graph-based traffic modelling and the Braess Paradox.
- Identified the main project objectives and the performance metrics required.

Phase 2: Graph Modelling and Algorithm Selection

- Designed a graph representation of junctions and road segments.
- Identified the required road attributes, including travel time, capacity, and traffic volume.
- Selected Dijkstra's algorithm, the BPR congestion function, and MSA for the simulation workflow.

Phase 3: Development of the Simulation Engine

- Implemented the traffic simulation workflow around graph-based routing and traffic assignment.
- Incorporated congestion-dependent travel-time calculations.
- Developed or integrated bottleneck detection and intervention-analysis functionality.

Work Progress

Phase 4: Testing and Validation

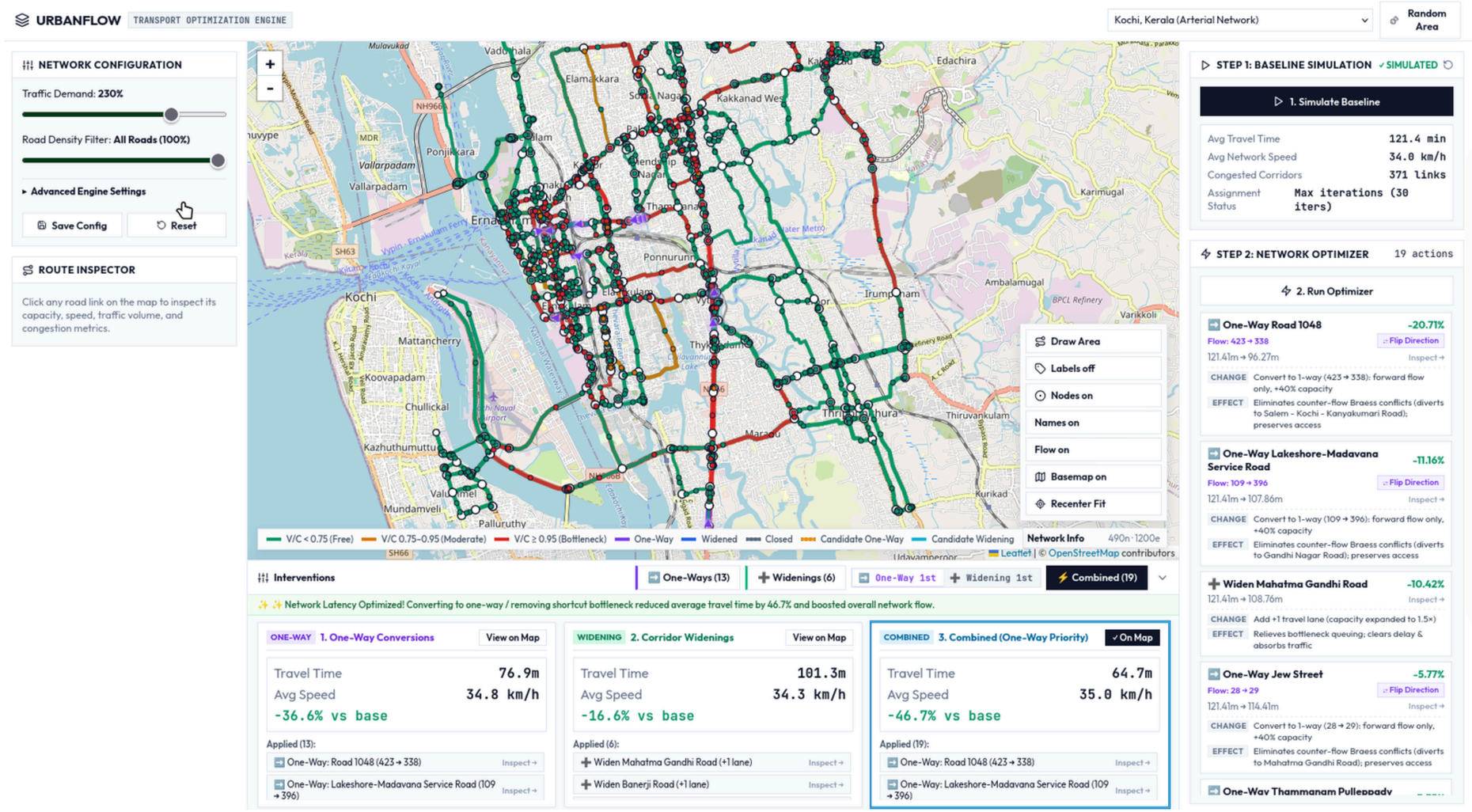
- Prepared a controlled Braess Paradox scenario.
- Compared traffic conditions before and after closing the middle road.
- Evaluated the resulting changes in traffic distribution and travel time.

Phase 5: Real-World Integration and Presentation

- Established the process for obtaining road-network data from OpenStreetMap.
- Applied the same workflow to the Kochi network.
- Developed the frontend/backend structure and prepared the system for demonstration.

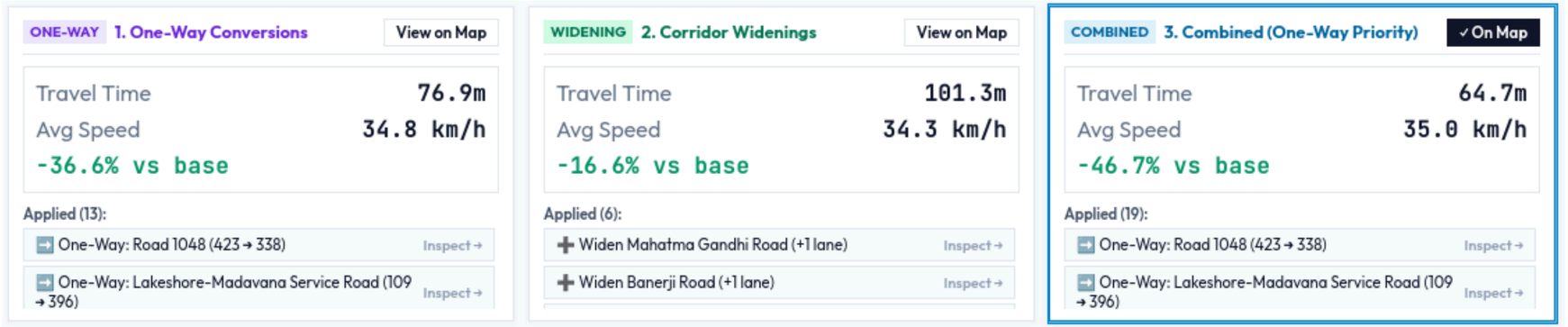
Implementation

UrbanFlow implements a graph-based traffic simulation engine along with a web dashboard for visualization using Kochi's real-world road network to evaluate congestion and baseline travel times and find optimizations for the infrastructure



Results

High Traffic (230%): Road optimization has a significant influence on congestion, as coordinated road changes help redistribute heavy traffic and reduce bottlenecks.



Low Traffic (100%): Road optimization has a smaller influence when traffic is lighter, since roads experience less congestion and fewer bottlenecks to resolve.



Sustainable Development Goals (SDGs)

SDG 11 – Sustainable Cities and Communities:

UrbanFlow supports sustainable urban mobility by optimizing road networks, reducing traffic congestion, and improving the efficiency of existing transport infrastructure.
Relevant target: 11.2 — Provide access to safe, affordable, accessible and sustainable transport systems.



SDG 9 – Industry, Innovation and Infrastructure:

UrbanFlow uses graph algorithms, congestion modelling, and simulation to evaluate road widening, one-way conversions, and other infrastructure improvements.
Relevant target: 9.1 — Develop reliable, sustainable and resilient infrastructure.



Conclusion

- UrbanFlow demonstrates how graph theory can be applied to the analysis and optimisation of urban traffic networks.
- The system represents junctions as vertices and roads as weighted edges, allowing traffic conditions to be analysed computationally.
- The BPR congestion function estimates how travel times change with traffic volume, while Dijkstra's algorithm calculates shortest-time routes.
- MSA iteratively updates traffic assignments to approximate user-equilibrium conditions.
- The controlled Braess Paradox scenario demonstrates that closing a road can sometimes reduce overall travel time rather than worsen it.
- The project extends the simulation workflow to the Kochi road network using OpenStreetMap data.
- Overall, UrbanFlow provides a simulation-based approach for investigating traffic bottlenecks and comparing infrastructure alternatives before real-world implementation.

Future Scope

- Real-Time Traffic Integration: Integrate live traffic information to improve the accuracy of travel-time estimates and traffic simulations.
- Machine Learning-Based Prediction: Use historical traffic data and machine learning models to predict congestion at different times of day.
- Larger Network Support: Improve performance and memory utilisation when analysing large metropolitan road networks.
- Advanced Traffic Optimisation: Explore additional traffic-assignment and network-optimisation techniques to identify more effective interventions.
- Environmental Impact Analysis: Estimate fuel consumption, vehicle emissions, and potential environmental benefits under different traffic scenarios.
- Enhanced Visualisation: Add detailed before-and-after comparisons, congestion heatmaps, and interactive scenario controls.
- Planning Reports: Generate reports containing performance metrics and intervention comparisons to support transportation planning.

References

[1] D. Braess, “Über ein Paradoxon aus der Verkehrsplanung,” Unternehmensforschung, vol. 12, pp. 258–268, 1968.
[2] Bureau of Public Roads, Traffic Assignment Manual. Washington, DC, USA: U.S. Department of Commerce, 1964.
[3] NetworkX Developers, “NetworkX documentation.” [Online]. Available: https://networkx.org/documentation/
[4] G. Boeing, “OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks,” Computers, Environment and Urban Systems, vol. 65, pp. 126–139, 2017.
[5] OpenStreetMap contributors, “OpenStreetMap.” [Online]. Available: https://www.openstreetmap.org/
[6] OSMnx Developers, “OSMnx documentation.” [Online]. Available: https://osmnx.readthedocs.io/
[7] FastAPI, “FastAPI documentation.” [Online]. Available: https://fastapi.tiangolo.com/
[8] React, “React documentation.” [Online]. Available: https://react.dev/
[9] Indian Roads Congress, IRC:106 — Guidelines for Capacity of Urban Roads in Plain Areas. New Delhi, India: Indian Roads Congress.